Memo To: Werner Buchholz

Subject: Address Interleaving Patterns for SIGMA.

I. Introduction

You recall we had a discussion a few days ago concerning the best method of interleaving addresses as additional memory boxes are added to SIGMA. The hope was that perhaps the SIGMA Timing Simulator could give an indication as to which method would result in the best system performance.

In looking through the results of various old simulator studies, I found that the necessary runs were already on hand for three different assumed machine configurations. From these one could compare the results obtained for different mixes of memory addresses for one typical problem. Certain assumptions mentioned below were made to simplify the analysis.

II. Assumptions

Three different methods of interleaving memory addresses were studied:

(1) all N memory boxes interleaved (called the "mixed" case below)

(2) Two memory boxes interleaved separately and the remaining N-2 boxes interleaved. (called the "2, X" case.)

(3) The memory boxes interleaved in pairs only (called the "2, 2, 2" case)

The following assumptions were made concerning the problems being done:

(1) The program can fit into two memory boxes (or one in the 1 box case) and is all in memory at once,

(2) The program is written so that data and instructions can be separated or kept together, but no further breakdown is allowed. That is, the program is located I D or I D but not I1 D1 I2 D2,

(3) The programs will have I and D in the order which would give the best performance even when they are being placed at random in the memory. That is if ID is the preferred order there are no programs in DI order,

(3) The programs are considered located in memory by one of two processes:

(a) The instructions and data separated and placed so as to give maximum performance. This is the technique which a human programmer would probably employ.

(b) The instructions and data each located at random in the memory. A supervisory program or Fortran-type coder might operate in this fashion particularly when multiprogramming is being used.

The total number of cases considered are Three

(1) Mixed Case randomly placed
(2) Mixed Case properly placed
(3) 2, X Case randomly placed
(4) 1, X Case properly placed
(5) 2,2,2 Case randomly placed
(6) 2, 2, 2 Case properly placed

~~graphs showing~~ The above six cases on a number of 2 μsec memories are shown in the following graphs. The graphs represent runs made in Jan 58, March 58, and Aug 58 with different assumed machine configurations and speeds.

Notice that cases (1) and (2) for the mixed case coincide since the memory is homogeneous and placement makes no difference.

III. Discussion of Random Location of Programs

The curves for random placement were obtained by assuming that the programs instructions and data were small and were each located randomly in the memory. The actual practice the fact that there is only one protected area in memory complicates the situation. There are two other ways of placing programs "at random" which a supervisory program might use, which can give quite different performance results. They are:

simply obtained by stripping off the low order bits of the address.

In the present bus system provision is made for attaching both half megabyte and two megabyte memories. The split can be made by a "range selection" on the high order bits of the address. If there is a 1 anywhere in one of these positions in the address it goes to one decoder and if more it goes to another. The two decoders then select the correct box for their respective series.

If the computer is limited to 2 decoders only the following combinations are possible:

No. of boxes	all mixed	2, X case	2, 2, 2 case	others
1	1			
2	2	2	2	
4	4	2,2	2,2	
6		2,4	2,2,2	
8	8		2,2,2,2	4,4
10		2,8	2,2,2,2,2	
12			2,2,2,2,2,2	4,8
14			2,2,2,2,2,2,2	
16	16		2,2,2,2,2,2,2,2	8,8



$34 = T_1 + 40 = T_{1C} + T_{10}$

$25 = T_2 + 29 = T_{2C} + T_{20}$

$T_{1C} = 43$

$\frac{34+26}{43} = \frac{60}{43}$

$T_{1d} = T_f$

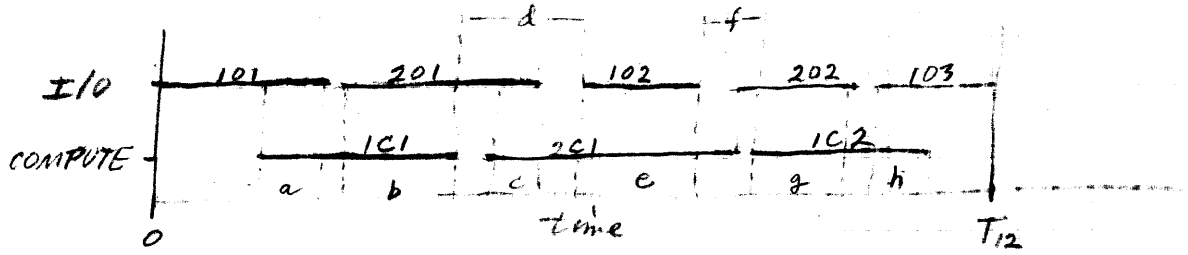
$\frac{69}{60}$

$T_{12} = \frac{T_1 + T_2}{f_{mult}} = \frac{T_{1C} + T_{10} + T_{2C} + T_{20}}{f_{mult} \cdot f_{or}} = \frac{1}{f_{mult} f_{or}} \left[\frac{T_{1C} + T_{2C}}{f_c} \right.$

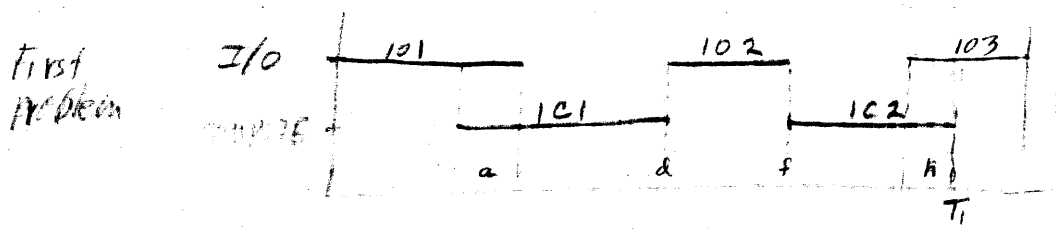
$R_1 = \frac{19}{21}$ $R_2 = \frac{16}{13}$

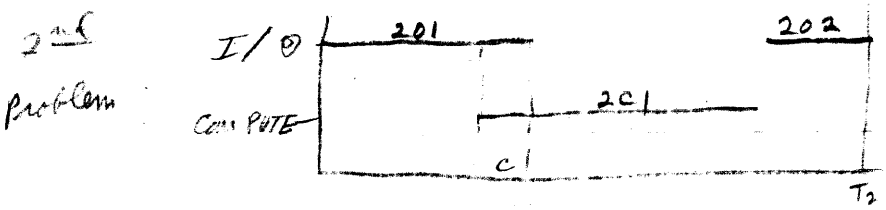
$\frac{35}{34}$

An example of the difficulty of giving a simple factor for I/O - compute overlap and multiprogramming overlap. Consider case of 2 problems multiprogrammed:



Computing and I/O are overlapped for the same problem in a, c, and h. They are overlapped between the problems in b, e, and g. The times d and f represent delays in problem 1. The effect of multiprogramming can be evaluated by examining the two problems separately.





The multiprogrammed I/O factor is given by

$$f_{MULT} = \frac{T_1 + T_2}{T_{12}}$$

The I/O-compute overlap factor is given by

$$f_{OV} = \frac{T_{1C} + T_{1O} + T_{2C} + T_{2O}}{T_1 + T_2}$$

In the above drawings $f_{MULT} = 1.4$, $f_{OV} = 1.2$, ... now The factors for I/O and compute may be applied to the individual times to give the values to the T24 Times,

$$(T_{1C} + T_{2C})_{704} = (f_{CALC})(f_{NOM})(f_{WDL})(f_{OP})\,(T_{1C} + T_{2C})_{SIG}$$

$$(T_{10} + T_{20})_{704} = (f_{RATE})(f_{FLEX})\,(T_{10} + T_{20})_{SIG}$$

Combining the four mod factors, we get the performance formula given before

$$P = \frac{1 + R}{\frac{1}{f_C} + \frac{R}{f_O}}$$

where R is calculated for the sum of the programs being multiprogrammed.

f_{CALC} :	80	to 24	50
f_{MEM} :	1.0	to 2	1.2
f_{WDL} :	1.0	to 8	1.1
f_{OP} :	1.0	to 1.3	1.2
f_{MOVE} :	1.0	to 4	1.1
f_{RATE} :	4.	to 40	10
f_{OV} :	1.0	to 2	1.2
f_{FLEX} :	1.0	to 1.2	1.05
R :	0	to 20	0.1

Tight fit

$f_c = \begin{matrix} 5 & 4 & 1 & 1 & 1 & 1 \\ 24 & 2 & 8 & 1.3 & 4 & 1.2 \end{matrix}$

$= \begin{matrix} 5 \\ 24. \\ 8000. \end{matrix}$

$f_c = \begin{matrix} 5 & 4 & 1 & 1 & 1 \\ 40 & 1.2 & 2 & 2 \end{matrix}$

$= \begin{matrix} 5 \\ 4 \\ 1.2 \end{matrix}$

Range of f $\begin{cases} P = \dfrac{1+20}{\frac{1}{24}+\frac{20}{4}} = \dfrac{21}{5.0425} = 4.17 \\ P = \dfrac{1+0}{\frac{1}{8}+0} = 8.000. \end{cases}$

$f_c = (50)(1.2)(1.1)(1.2)(1.2)(1.1)$

$= 103.$

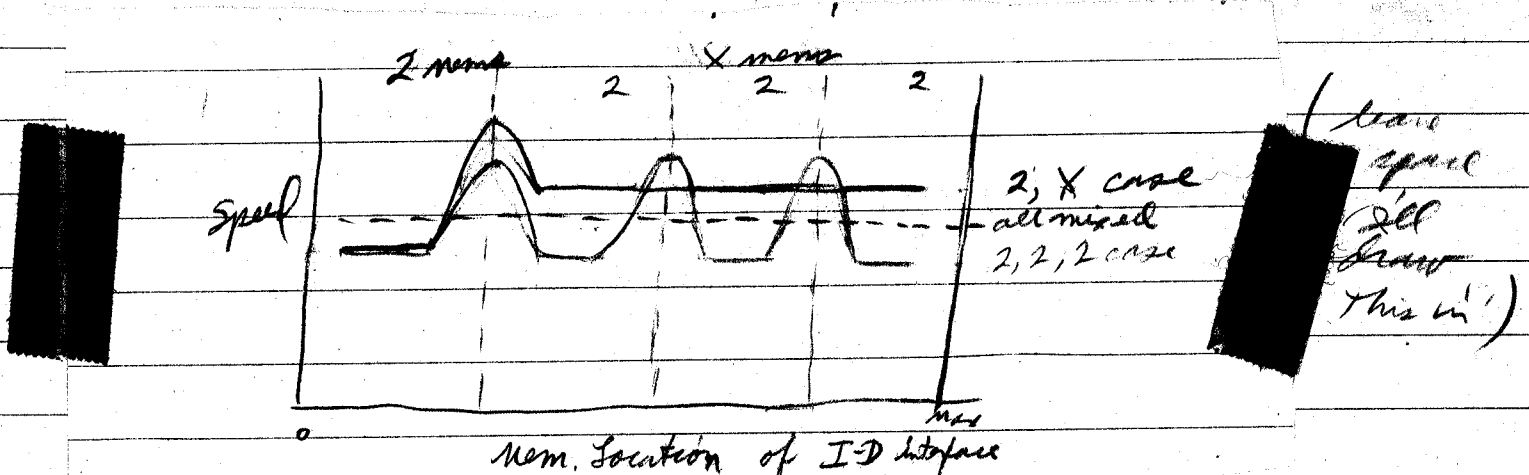
$f_c = (10)(1.05)(1.2)(1.1)$

$= 14$

$P = \dfrac{1+0.1}{\frac{1}{103}+\frac{1}{14}} = \dfrac{1.2}{.0097+.0071}$

$P = \dfrac{1.2}{.0168} = 72.$

(1) The data and instructions for a given program might always be kept next to each other in the unprotected area and located anywhere in memory. The resulting performance would show a complicated pattern depending on their location in memory.



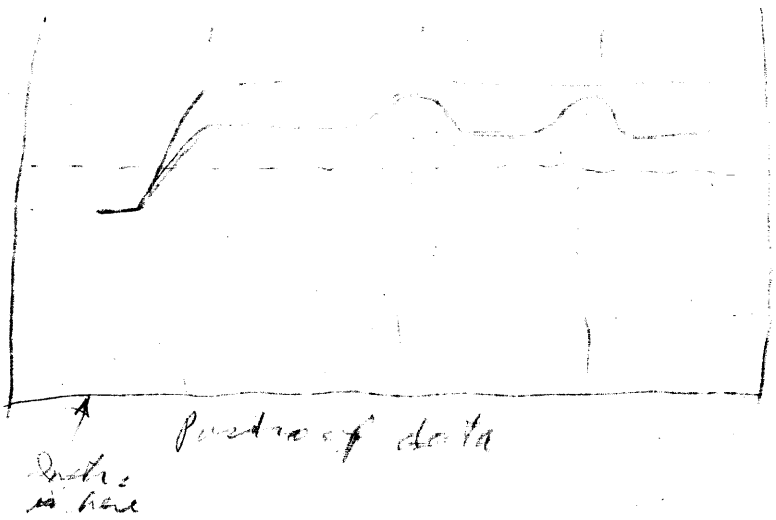
The exact shapes of the curves about the maxima depend on (1) the amounts of data and instructions compared to the size of a memory box, (2) the location within the instruction area of the active inner loops, and (3) the location within the data area of the most active data.

The result is that even though we know the values of the peaks and valleys for the above curves there is no single average value which will hold for the random placement of any program.

(2) The programming convention of "read-only" instructions might be enforced so that the supervisory program could leave the instructions in the protected area of memory.

This way the instructions would be prepared, placed ~~with the~~ unprotected area, could be placed anywhere else in memory,

This arrangement gives another benefit, it takes extra ~~but~~ can be better compile performance.



(Delete this)

The indication is that the expansionary program could improve the computer performance ~~appreciably~~ by being sure to separate the data and instructions in some such manner

V. Discussion of Memory Interleaving.

The Simulator can intermix any number of memory boxes in several patterns since the box can be selected by taking the address Mod the number of boxes mixed. In the real machine we cannot (even) consider the time penalty which a true division would take. The result is that we can intermix only those powered number of boxes. The Mod function is then

Exact shape of  depends on size of I + D compared to 16K.

→ It also depends on location of inner loop of code and highly active data locations, etc,

8,8	4,8	2,8	4,4	2,4	2,2	2 –

If the two cores each have the same number of boxes (e.g. 4, 4) only one decoder is necessary, while the decoding can be done before the selection.

The following table lists the possible combinations of boxes, the number of decoders and storage selectors needed, and the approximate speed for each combination (applied to the "Recommended Speed", "properly placed" cores.)

Number of Boxes	combination	No. of decoders	No of selectors	Approx. Speed
1	1	0	0	42.
2	2	1	0	57.
4	4	1	0	64.
	2, 2	1	1	70
6	2, 4	2	1	72.
	2, 2, 2	1	2	70.
8	8	0	0	67.
	4, 4	1	1	72.
	2, 2, 4	2	2	72.
	2, 2, 2, 2	1	3	70
10	8, 2	2	1	72
	4, 4, 2	2	2	72
	4, 2, 2, 2	2	3	72
	2, 2, 2, 2, 2	1	4	71
12	8, 4	2	1	72
	4, 4, 4	1	2	72
	2, 2, 4, 4	2	3	72
	2, 2, 2, 2, 4	2	4	72

(cont.) No. of Boxes	Combination	No. of decoders	No. of selectors	Approx. Speed
14	8, 4, 2	3	2	72
	8, 2, 2, 2	2	3	72
	4, 4, 4, 2	2	3	72
	4, 4, 2, 2, 2	2	4	72
	4, 2, 2, 2, 2, 2	2	5	72
	2, 2, 2, 2, 2, 2, 2	1	6	71
16	16	0	0	71
	8, 8	1	1	72
	8, 4, 4	2	2	72
	8, 4, 2, 2	3	3	72
	8, 2, 2, 2, 2	2	4	72
	4, 4, 4, 4	1	3	72
	4, 4, 4, 2, 2	2	4	72
	4, 4, 2, 2, 2, 2	2	5	72
	4, 2, 2, 2, 2, 2, 2	2	6	72
	2, 2, 2, 2, 2, 2, 2, 2	1	7	72

Results

(1) Above 8 boxes there is little speed preference between any of the possible combinations.

(2) For 1 and 2 boxes there is no choice: For 4 boxes there is a distinct preference for 2, 2 rather than 4. For 6 boxes 2, 4 holds a 3% lead over 2, 2, 2. For 8 boxes 4, 4 or 2, 2, 4 are better than 2, 2, 2, 2 which in turn is better than 8.

(3) another point to observe is that the 2, X case and the mixed case can have only certain combinations of box numbers, in particular:

mixed case: 1, 2, 4, 8, 16

2, X case: 4, 6, 10

(4) The 2, 2, 2 case has the advantage that only one decoder is required but the number of range selectors are the largest.